Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Getting Started

Introduction to Computer Programming

Management and Artificial Intelligence

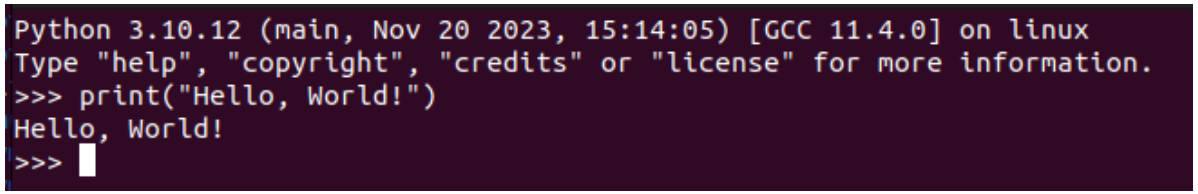
LUISS

How and Where

How can we run Python code?

Two approaches:

1. Interactive shell:

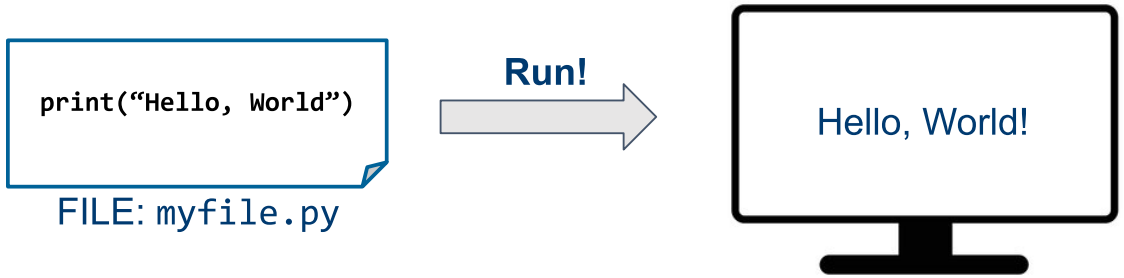


- workflow:
  - start the interactive Python shell
  - write one line of code in the prompt after the symbols `>>>`
  - push enter to execute it and get the results
  - repeat as much as needed
- **not very user-friendly**
- appropriate only for quick tests
- nothing is saved: you will lose your code when exiting the shell!

How can we run Python code? (cont'd)

Two approaches:

2. Write the code into a textual file and then execute it:



- workflow:
  - write the code into a textual file with extension `.py`
  - execute the file with the Python VM
- **This is what you want to do!**
- Still *iterative* approach: write the code, check if it works, refine the code, ...

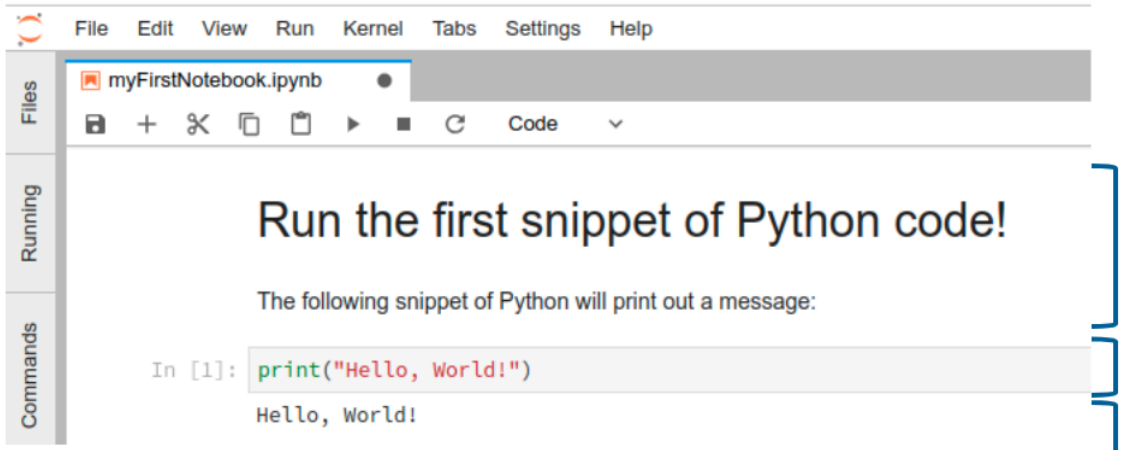
The code can be very cryptic...
Can we add some notes around it?

It could be hard to remember why we have written a piece of code, what are its effects and downsides.

It is often convenient to organize our code into "sections" and then add some text to describe what the code is expected to do. This is called:

Jupyter Notebook

Using the environment **JupyterLab** or with a proper Visual Studio Code setup, we can create a new notebook:



Text section describing what we is relevant for us (humans!)

snippet of code

result of the code (after running it with the play button)

Where do we run our code/notebooks?

1. **Local setup**: your computer!

- you have to install Python and all the other bits
- limited by your computational power

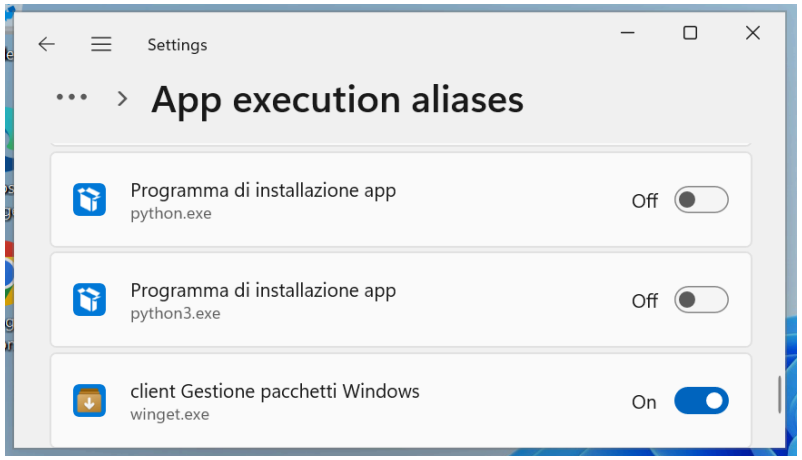
2. **Cloud setup**:

- no need to install everything
- limited by your money

Local Setup

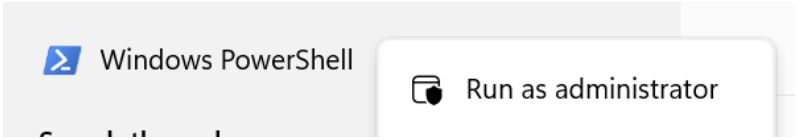
[**Only on Windows 11**]
Disable Windows Store Aliases

1. From the start menu, search and then open *Manage app execution aliases* (*Alias di esecuzione delle app*)
2. Disables aleases for *python.exe* and *python3.exe*:

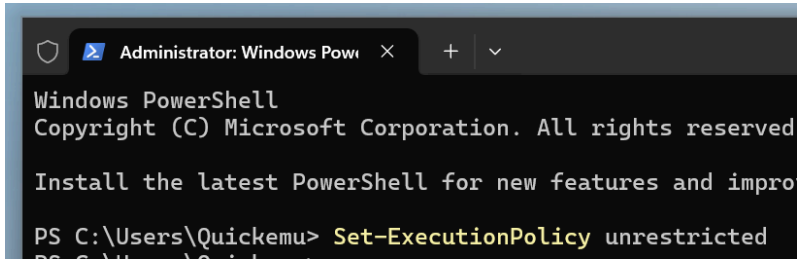


[Only on Windows 11]
Enable script cmdlet

1. From the start menu, search *Windows Powershell*, then using the right-click select *Run as administrator*:

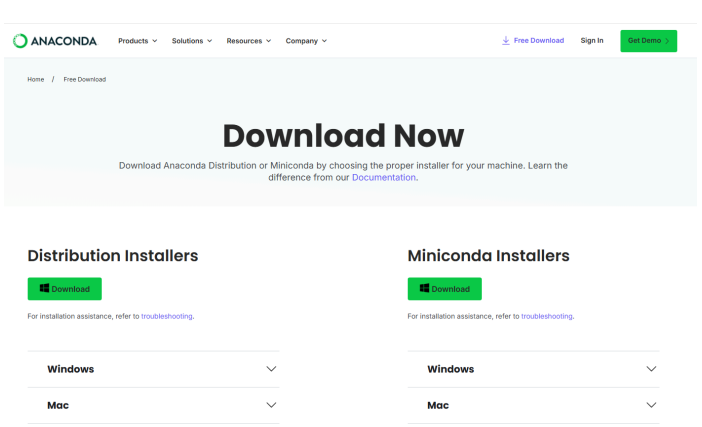


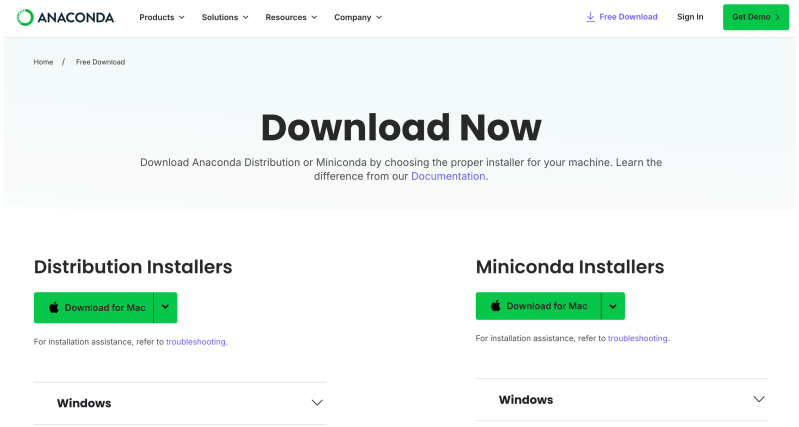
2. Type `Set-ExecutionPolicy unrestricted`, then run it and reply *y* (or "s" if your Windows is in italian) :



Installing Python via MiniConda

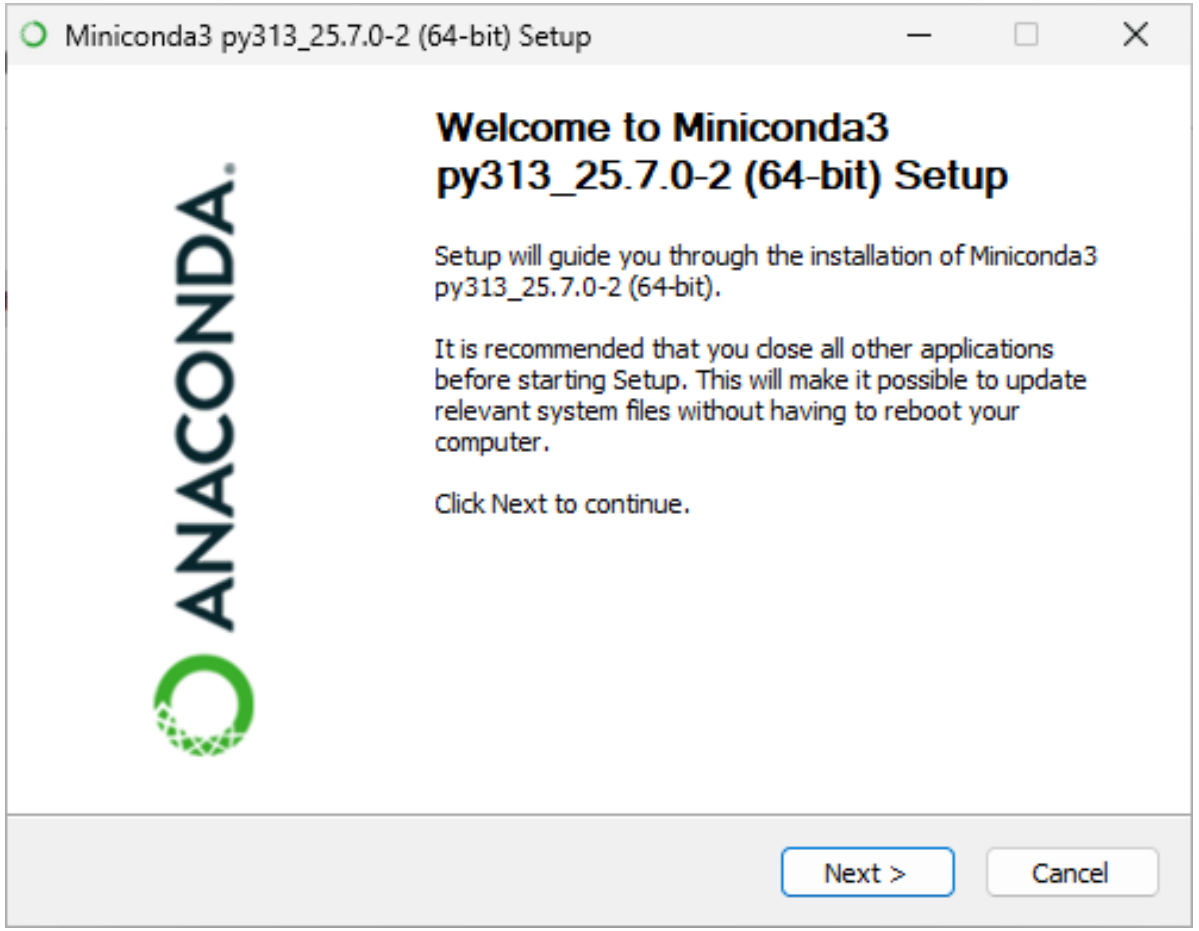
1. Open your web browser (e.g. Google Chrome, Microsoft Edge, Apple Safari, Mozilla Firefox) and go to https://www.anaconda.com/download/success
2. Click on the *Download* (for Windows) / *Download for mac* (for Mac) button under *Miniconda Installers* to get the installer





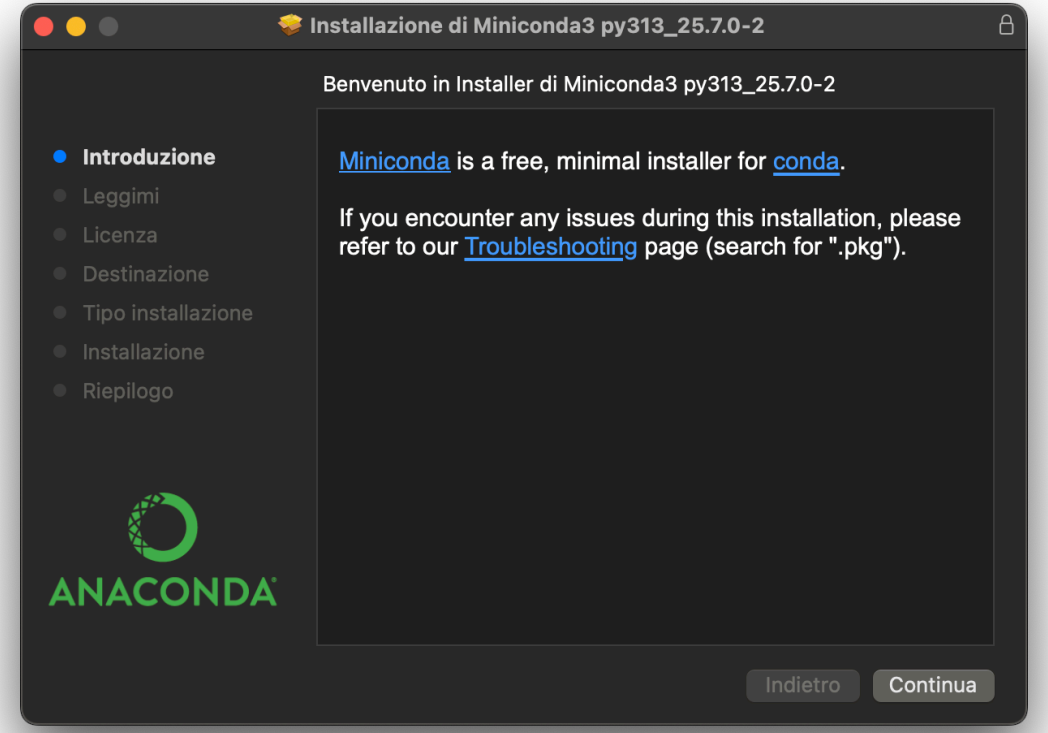
Launch the installer

Double click on the downloaded file (e.g. on Windows) to start the setup:



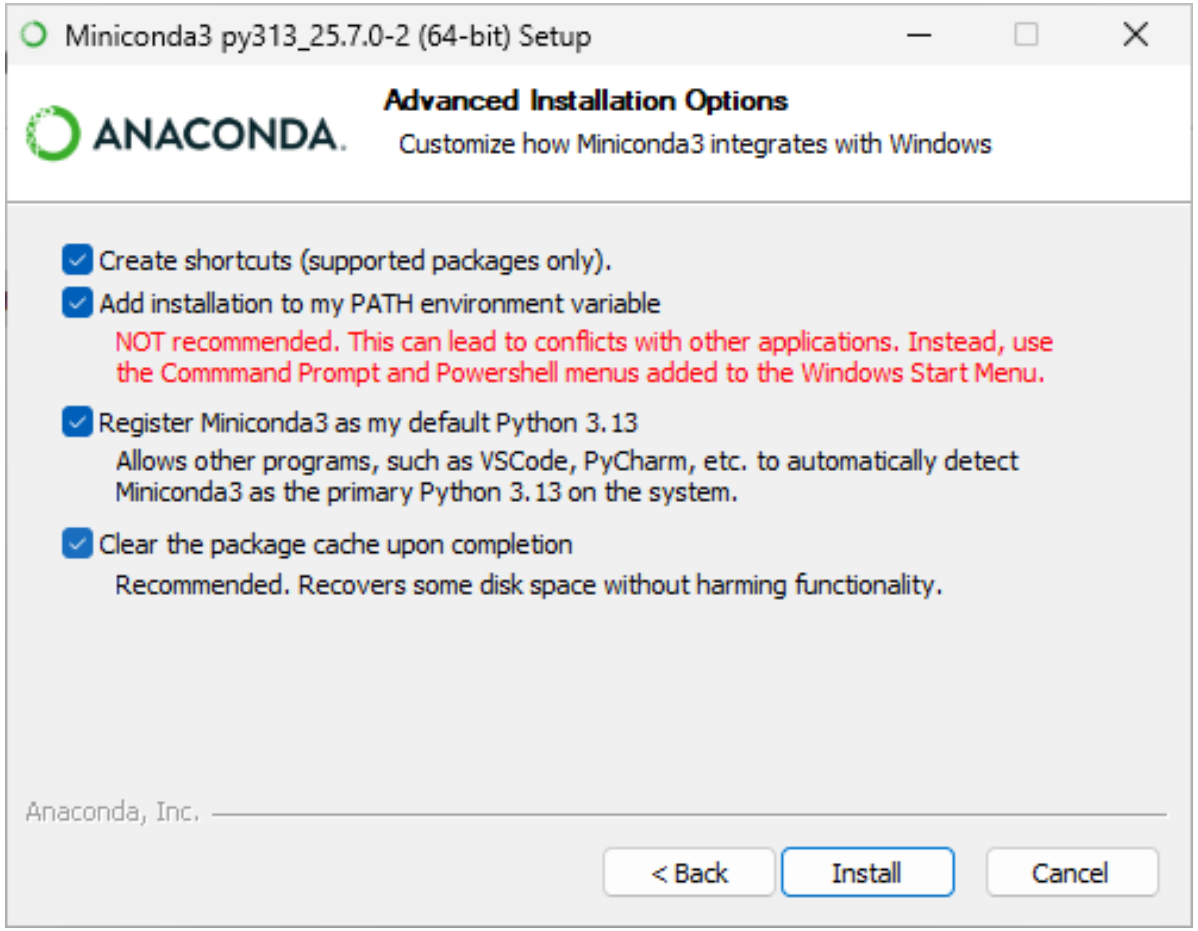
Launch the installer (cont'd)

Similarly on Mac OS X:



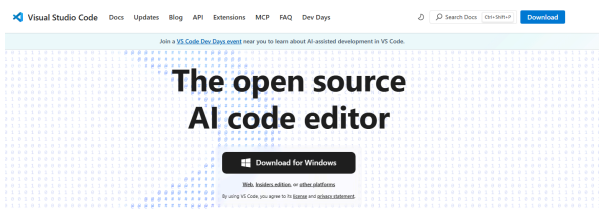
Launch the installer (cont'd 2) [Windows only]

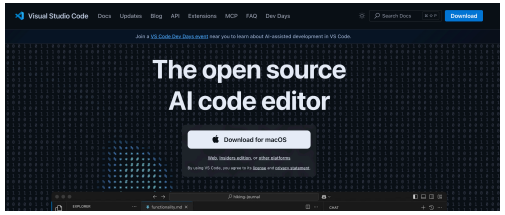
- In the *Advanced Installation Options* view, check "Add installation to my PATH environment variable", ignore warnings



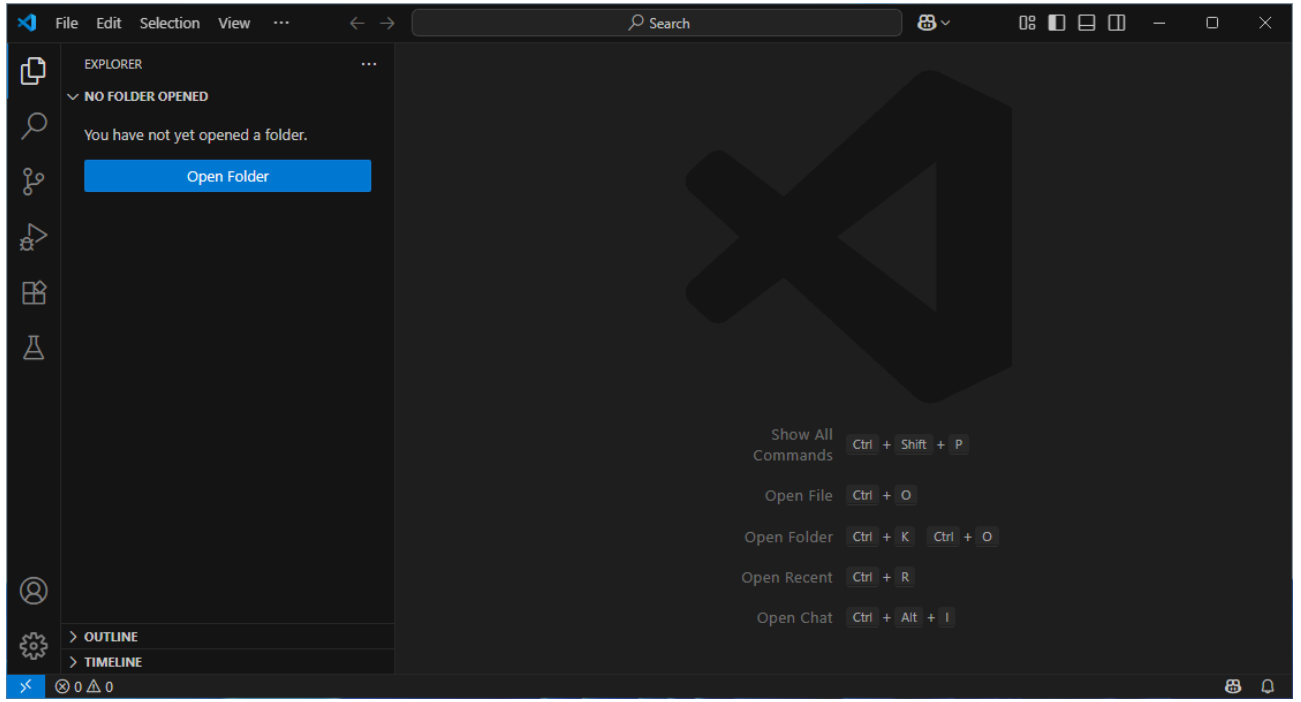
Install a (good) code editor: **Visual Studio Code**

1. Visit: **https://code.visualstudio.com**
2. Click on the button "Download for Windows" (if you have windows) or "Download for Mac" to start the download [check needed]



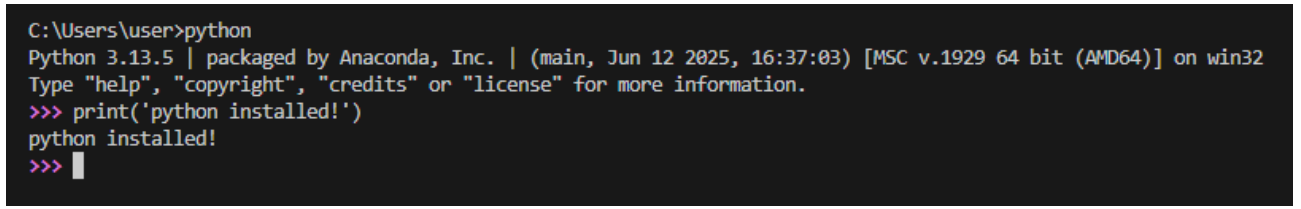


Start **Visual Studio Code**



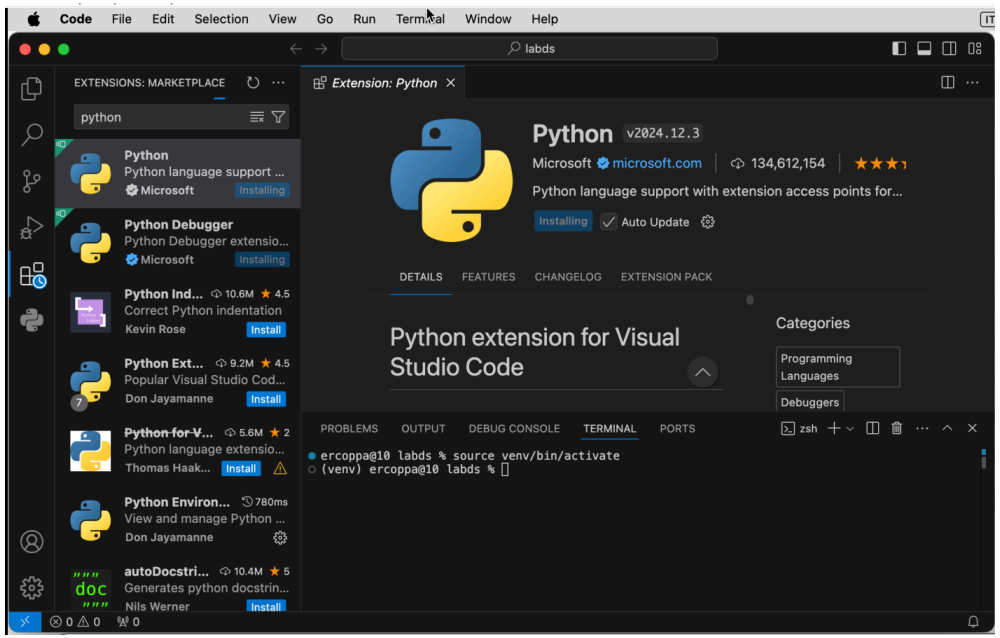
Test python installation

1. In VS Code start the terminal: ***Terminal > New Terminal***
2. Type `python` to open the interactive shell, a prompt input will appear with symbols `>>>`
3. Type the code `print('python installed!')` and hit enter, you will see the shell printing on screen "python installed!"



VS Code extension: Python integration

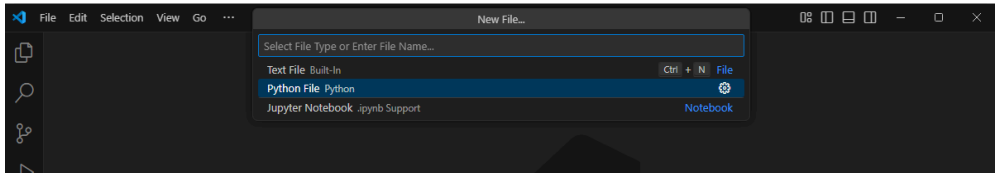
1. From the side bar, open the ***Extension*** tab
2. Search for ***Python***
3. Install the extension



Create and run a python file in VS Code

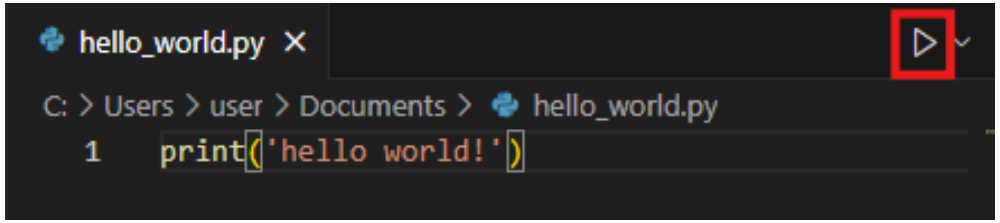
1. In Visual Studio Code ***File > New File***

2. Select ***Python File***



3. In the opened file type `print('hello world!')` and save the file

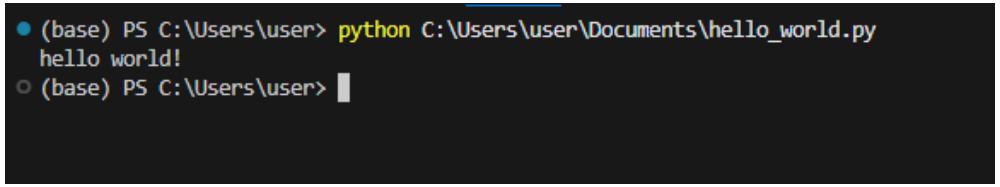
4. Click on the **play button** on the top right corner and the code will be executed in a new terminal



Create and run a python file in VS Code (cont'd)

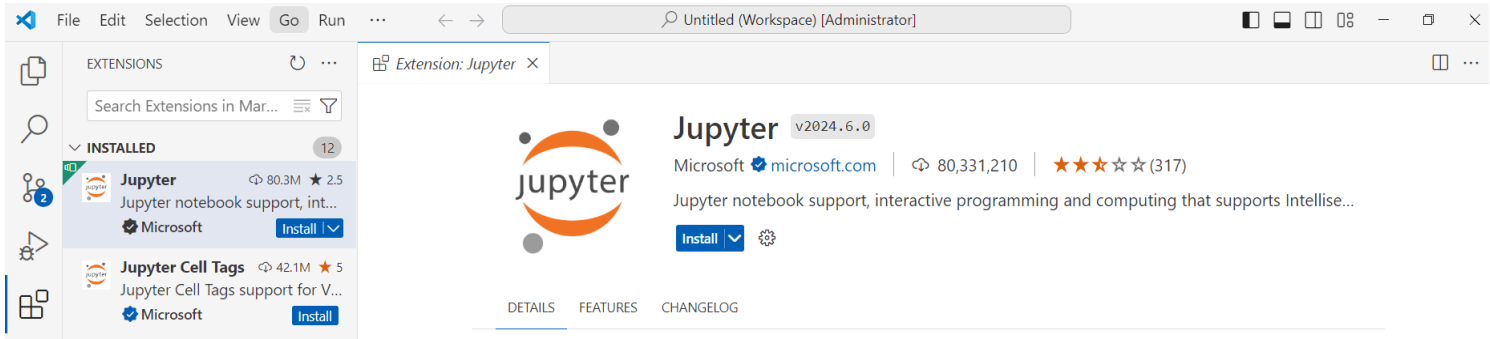
When you hit play, VS Code is just running a command line in the terminal. That's another way to run python files:

1. In Visual Studio Code open the terminal ***File > New Terminal***
2. Type `python <full-path-to-the-file>` and hit enter
   (e.g. on Windows if you have a file named "hello_world.py" in the Documents folder)



VS Code extension: Jupyter

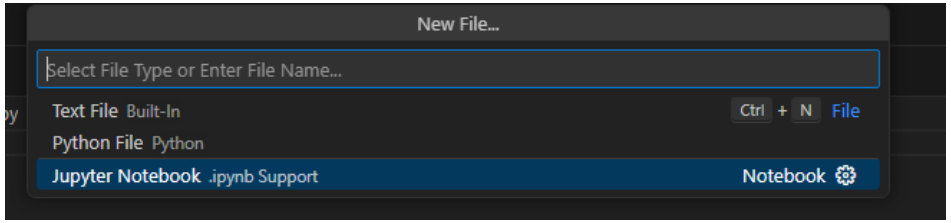
1. From the side bar, open the ***Extension*** tab
2. Search for ***Jupyter***
3. Install the extension



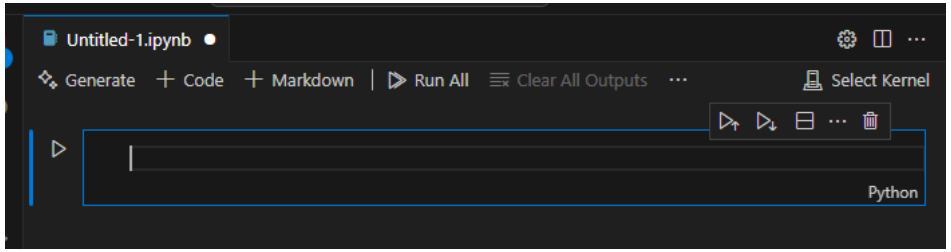
Create a new jupyter notebook

1. In Visual Studio Code ***File > New File*** tab

2. Select ***Jupyter Notebook***

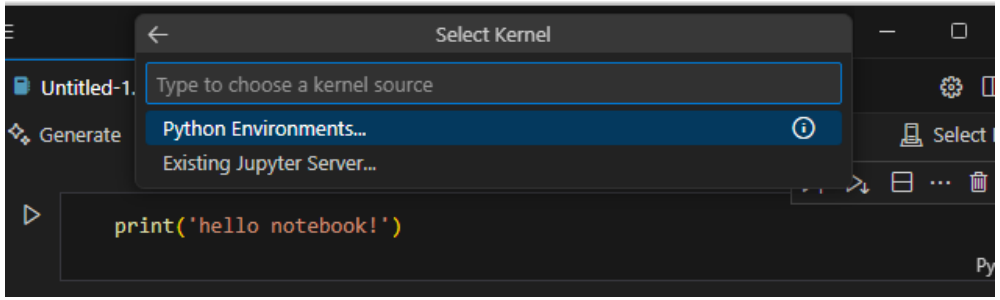


3. A new notebook will open



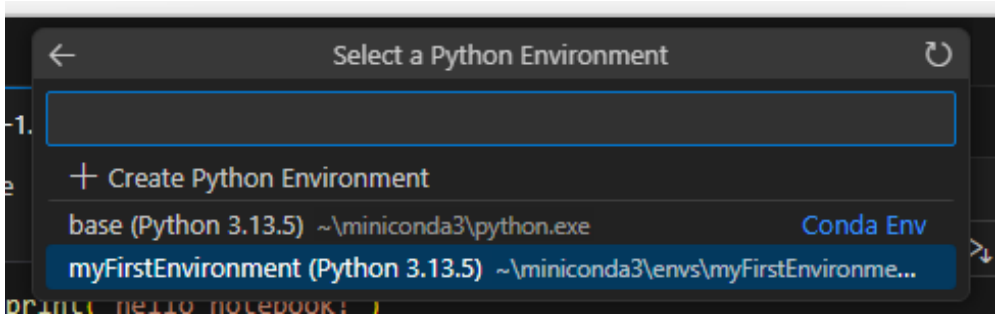
Running code in Jupyter notebook

1. In Visual Studio Code ***File > New File > Jupyter Notebook***
2. When you create a new notebook, the new file contains a new code cell.
3. In the code cell type `print('hello world!')`
4. In the top right corner click on **Select Kernel > Python Environments...**

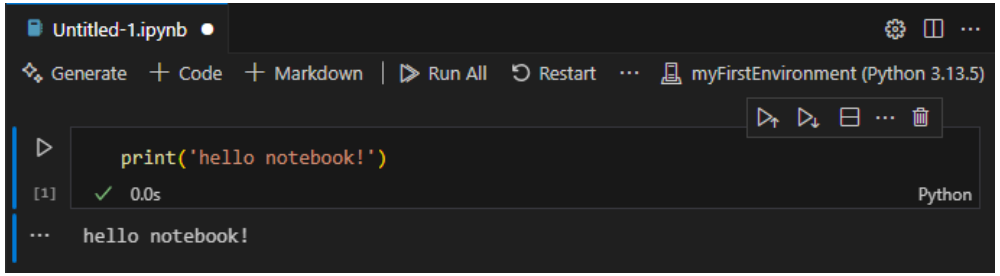


Running code in Jupyter notebook (cont'd)

5. Choose the python environment you've created for your project



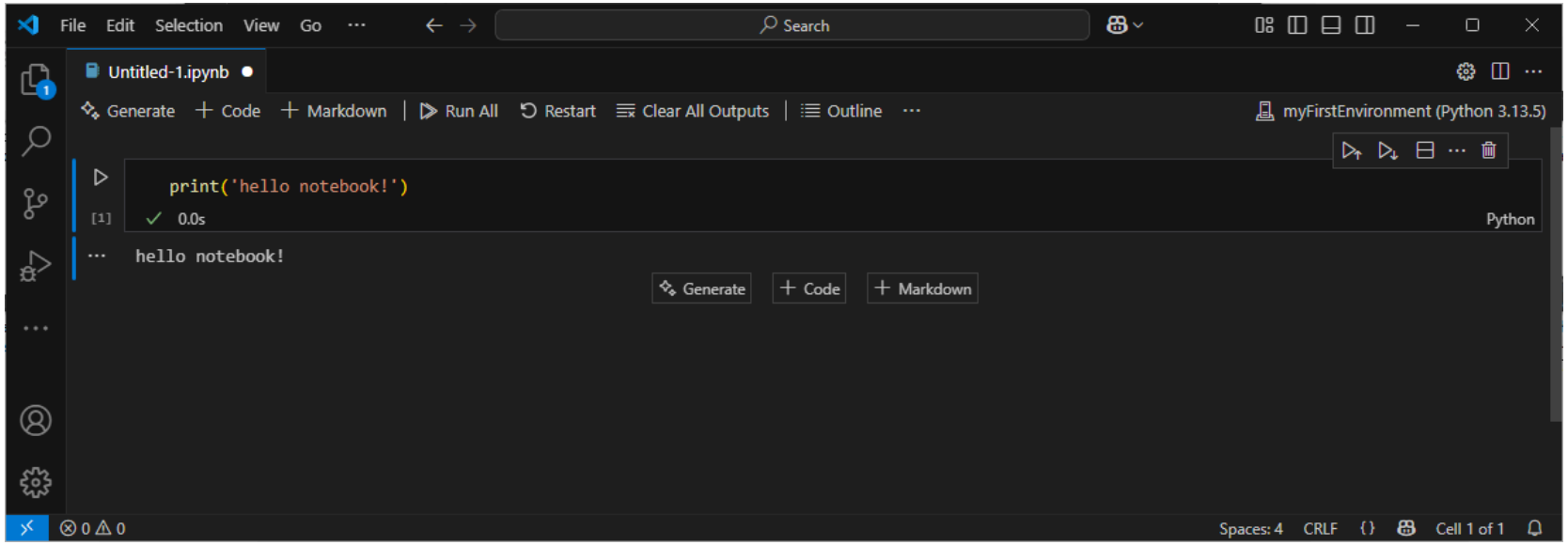
6. Click the play button on the left side of the cell. The code will run and the result will appear under the code cell



Working with jupyter

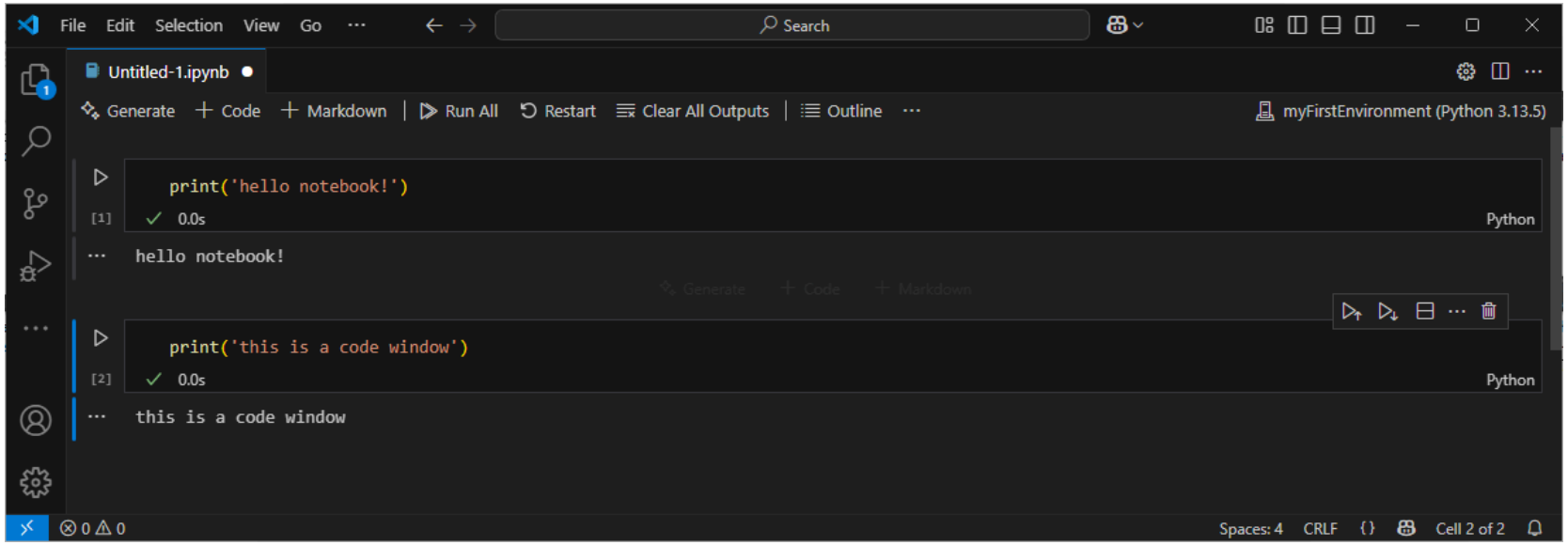
In a Jupyter notebook you can add two different types of cell

1. Code - to write code to run
2. Markdown - to add text and format it



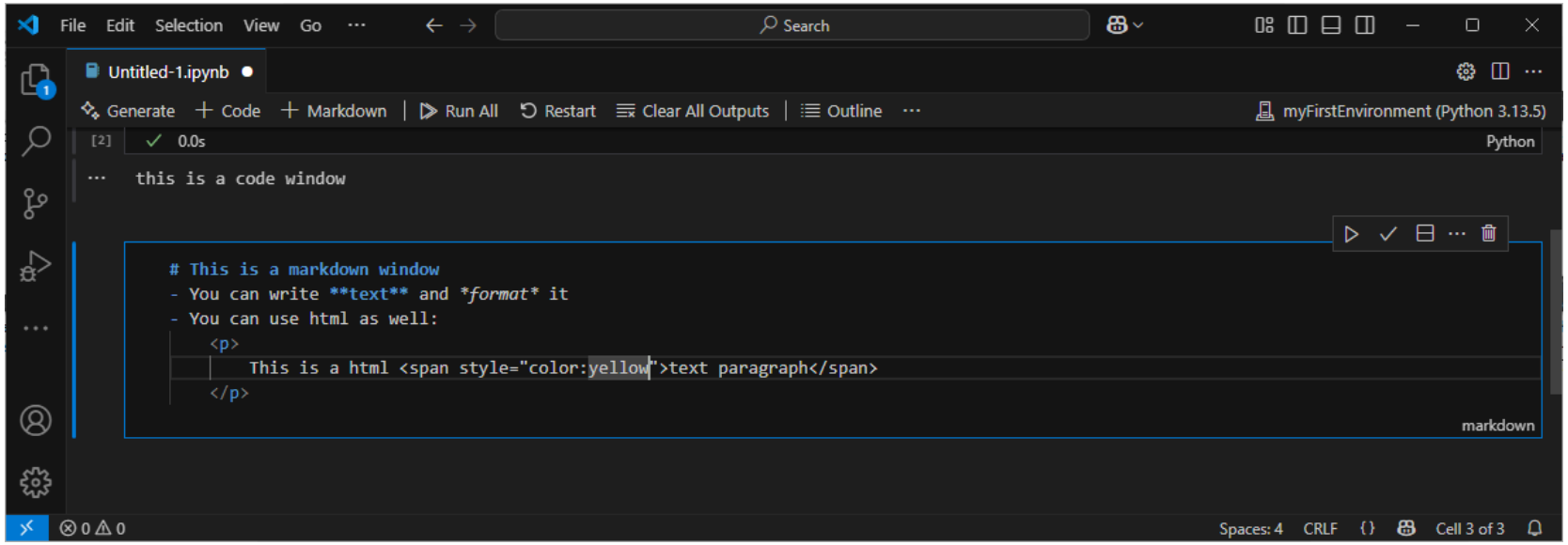
Working with jupyter - Code cell

1. Click on the "+ Code" button
2. Write python code
3. Run it (play button or *shift* + *enter*)



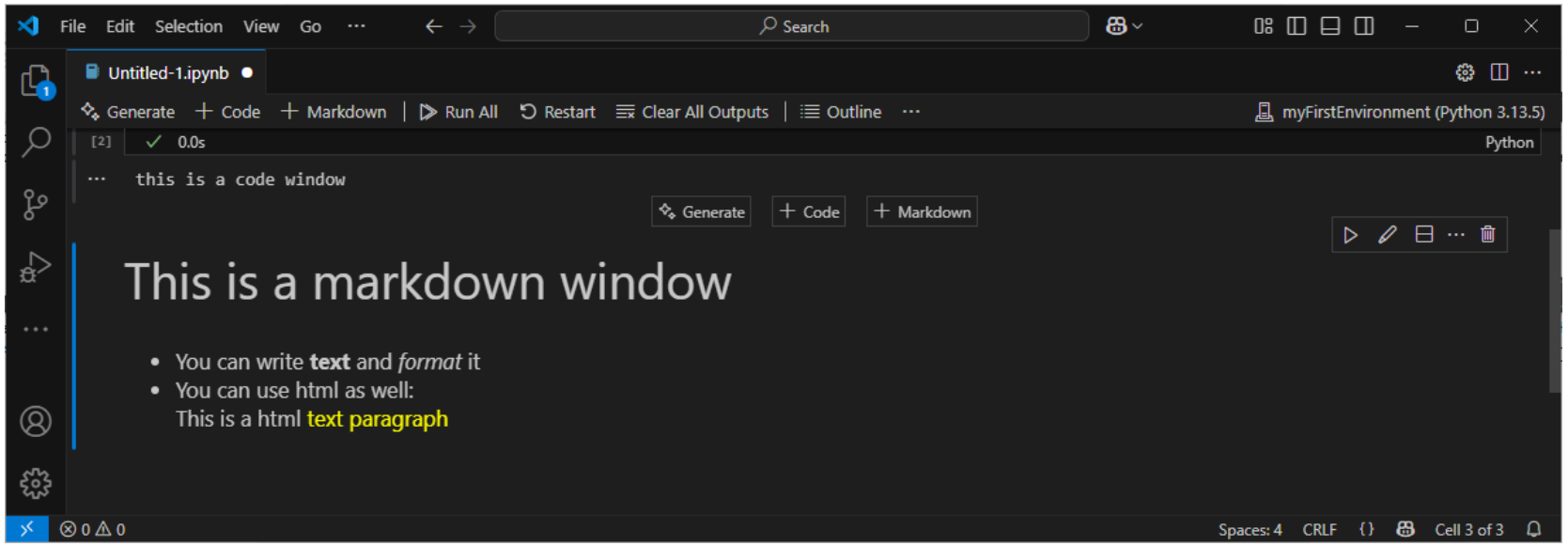
Working with jupyter - Markdown cell

1. Click on the "+ Markdown" button
2. Write text and format it with markdown symbols
3. You can also use html



Working with jupyter - Markdown cell (cont'd)

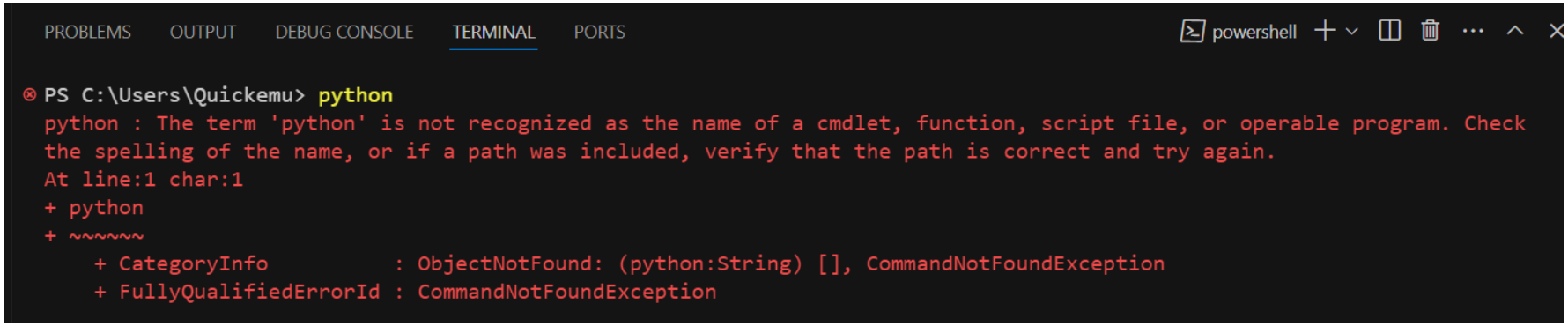
4. Confirm it (Check symbol on the top right or *shift* + *enter*) to see the result



Troubleshooting on Windows

Troubleshooting on Windows: `python is not recognized`

If, when when running `python` in the terminal of VSCode, you get the following error:

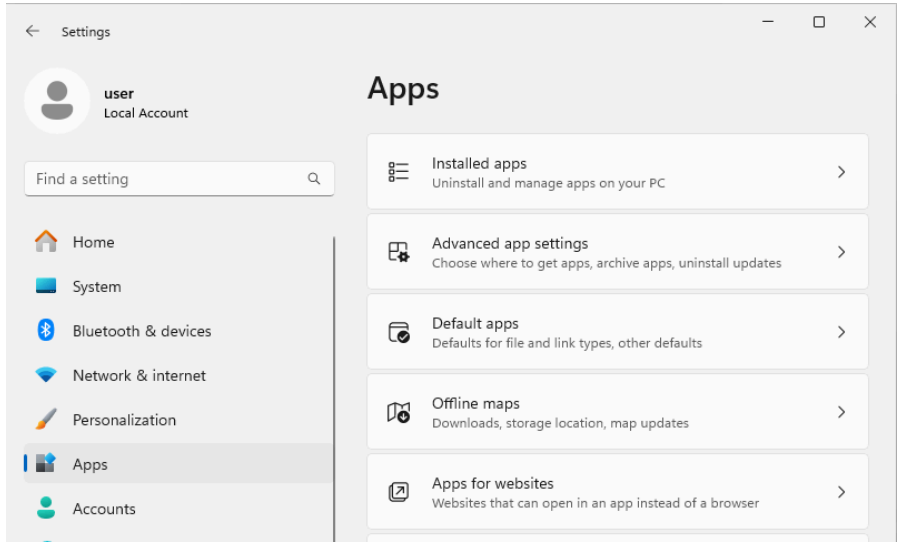


Then, you have to:

Troubleshooting on Windows: `python is not recognized` (cont'd)

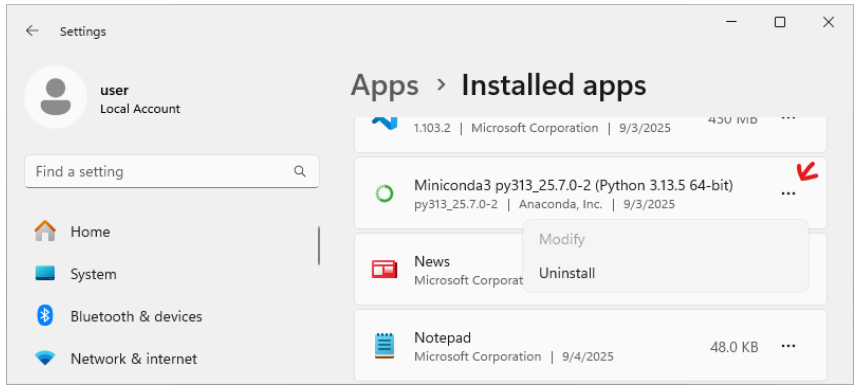
STEP 1 Uninstall Miniconda:

1. Click on **Start > Settings**
2. Select **Apps** from the left menu and then click **Installed apps**



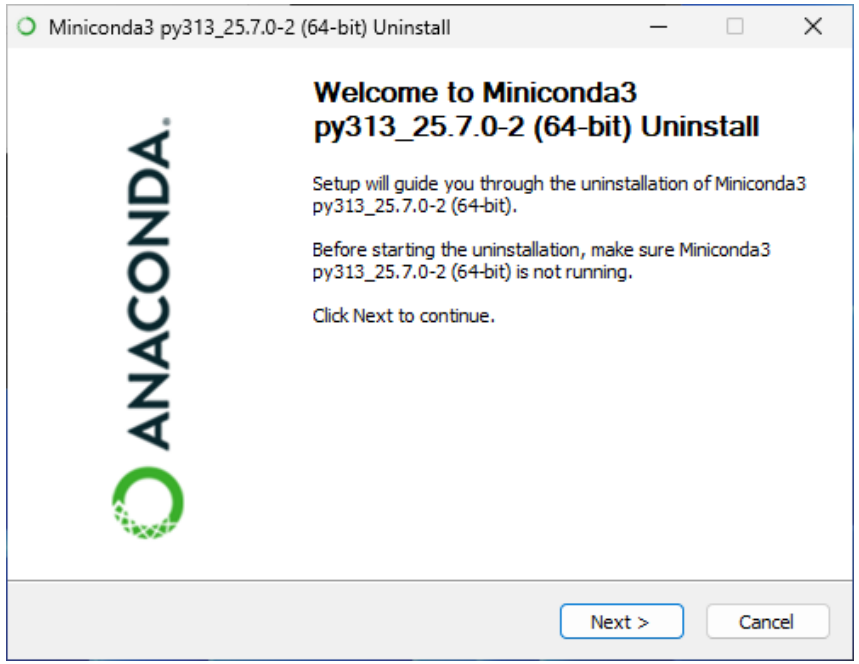
Troubleshooting on Windows: `python is not recognized` (cont'd - 2)

3. Look for Miniconda, click on **... > Uninstall**



Troubleshooting on Windows: `python is not recognized` (cont'd - 3)

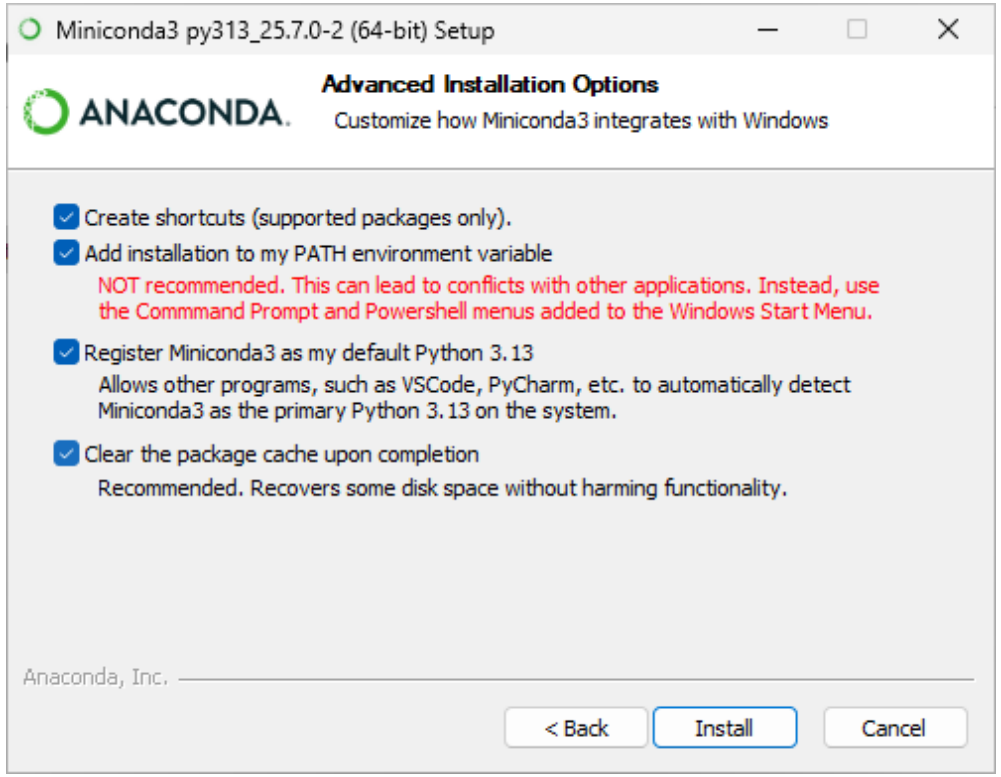
4. Run the conda uninstaller



Troubleshooting on Windows: `python is not recognized` (cont'd - 4)

STEP 2 Reinstall miniconda properly:

1. Start the miniconda installer
2. Click next and accept license terms
3. In the *Advanced Installation Options* view, check "Add installation to my PATH environment variable", ignore warnings. Then continue the setup.


Miniconda3 py313_25.7.0-2 (64-bit) Setup
ANACONDA.
Advanced Installation Options
Customize how Miniconda3 integrates with Windows
Create shortcuts (supported packages only).
Add installation to my PATH environment variable
NOT recommended. This can lead to conflicts with other applications. Instead, use the Commmand Prompt and Powershell menus added to the Windows Start Menu.
Register Miniconda3 as my default Python 3.13
Allows other programs, such as VSCode, PyCharm, etc. to automatically detect Miniconda3 as the primary Python 3.13 on the system.
Clear the package cache upon completion
Recommended. Recovers some disk space without harming functionality.
Anaconda, Inc.
< Back
Install
Cancel

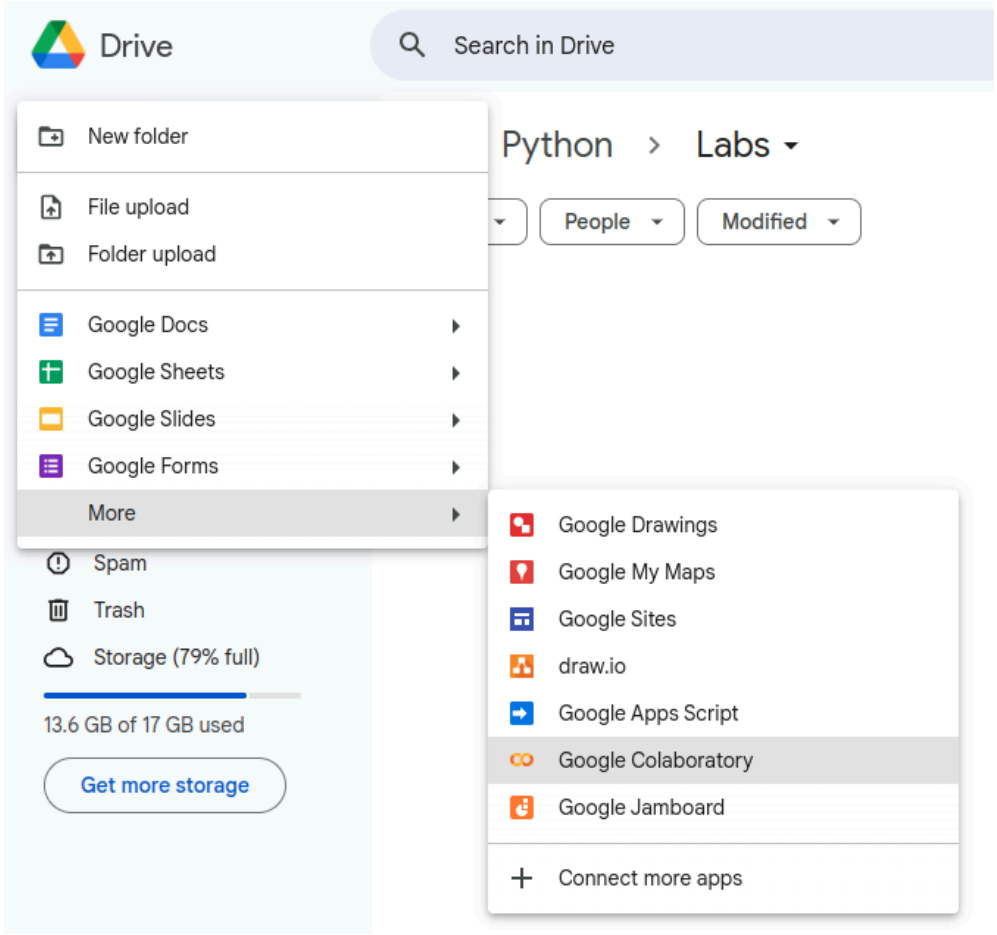
Cloud Setup

Google Colab, i.e., the lazy approach

- Platform from Google
- **Google Account is required**
- Free of charge to use for the basic plan
- Write code in the browser (no extra is needed)
- Code executed on Google cloud servers
- Low/medium performance, privacy issues
- **No need to install anything**

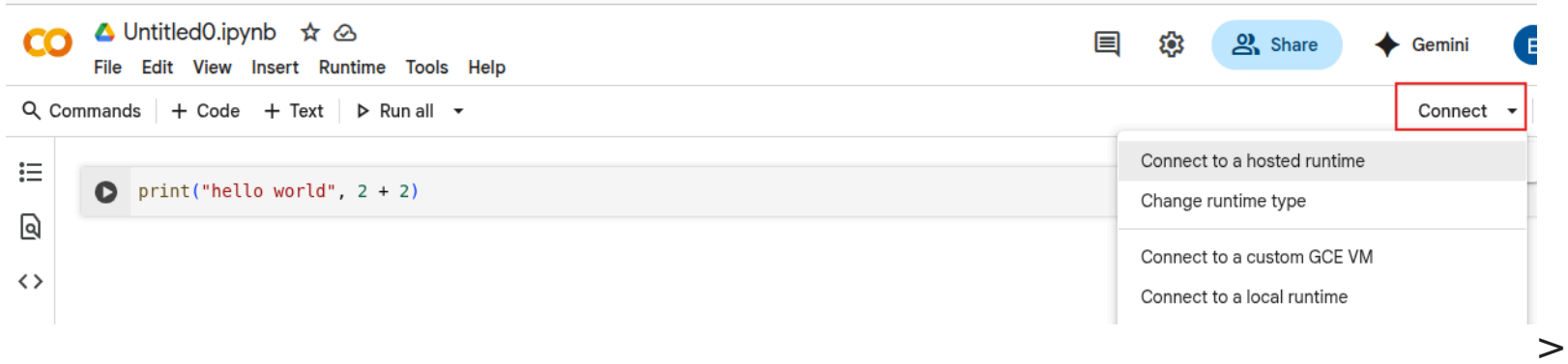
Google Colab, i.e., the lazy approach (cont'd)

- From **Google Drive**



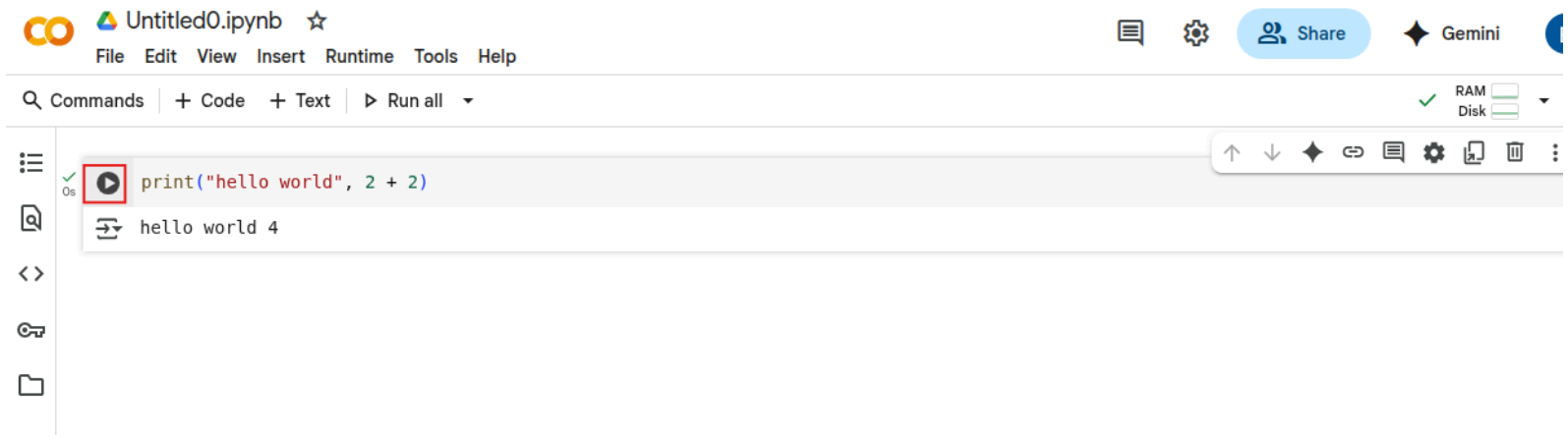
New Colab notebook

- A new notebook will open in colab
- Use as usual
- Before running you must connect to a runtime



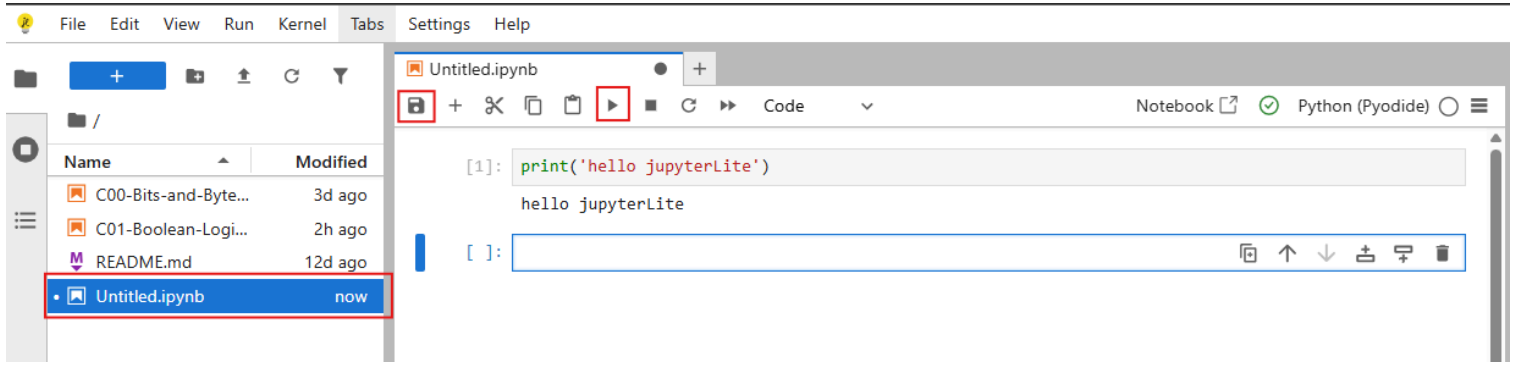
New Colab notebook (cont'd)

- Once connected, write code and run code in cells pressing the play button on the left side of the cell



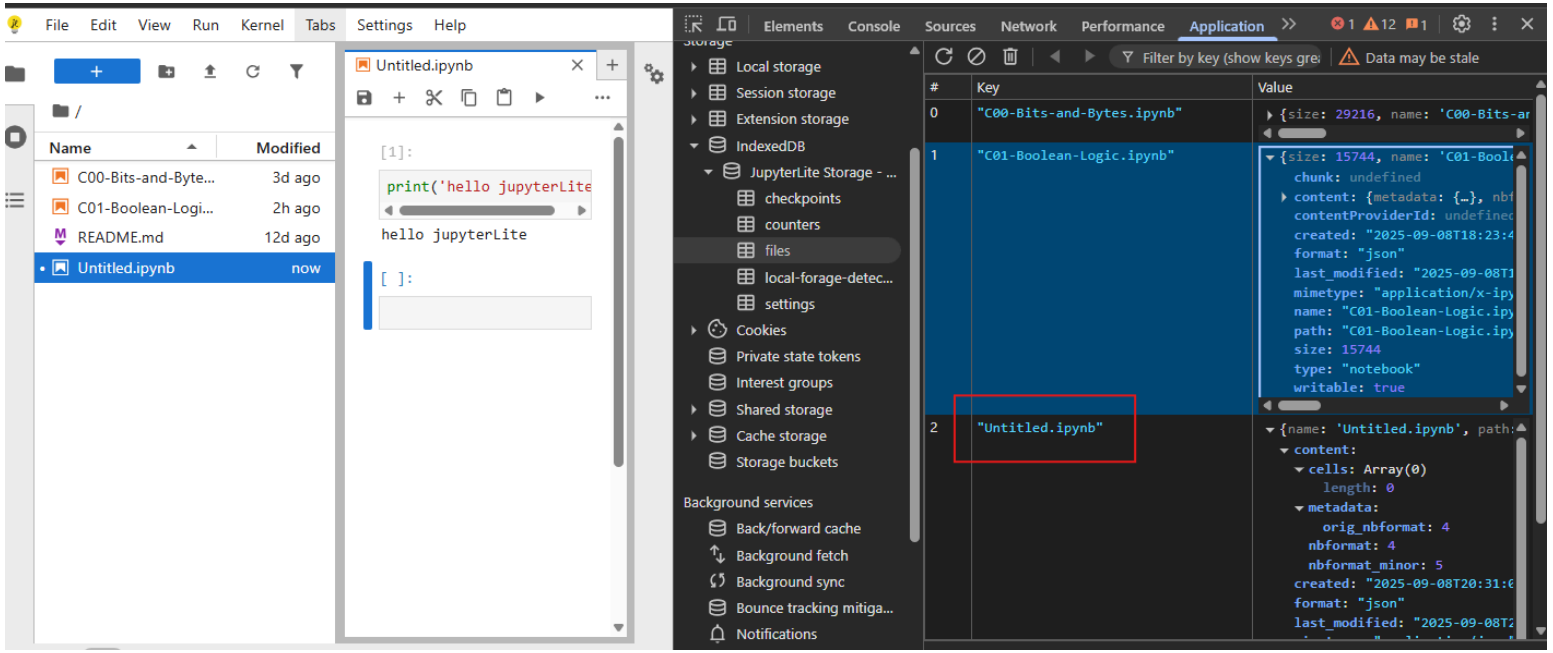
JupyterLite

- https://ercoppa.github.io/jupyterlite/
- **It does not require an account**
- Add code and cells
- Run with play button (or *shift + enter*)
- Save your file



JupyterLite (cont'd)

- Your files are saved in the browser's cache
- **You will lose them when you'll delete your "cookies and other site data"**



Environments

Environment: local setup with VSCode

Use VSCode:

- Quickly write and execute python files (`.py`)
- Quick access to the local terminal
- Write and execute notebooks (`.ipynb`). Some advanced features of JupyterLab (that we do not need) may have limited support.
- Good editor with several extensions (integrations with third-party services)
- Everything runs on your machine

This is the environment that we suggest you to use.

Environment: cloud setup with Colab

Use Google Colab:

- You can work from any computer. Even your tablet (although, it is a bit inconvenient!)
- Write and execute on the cloud notebooks (.pynb). Some advanced features of JupyterLab (that we do not need) may have limited support.
- Basic editor
- Everything must be saved on Google Drive
- **Not adequate in future courses where you need a bit of computational power** (assuming you do not want to pay Google).

We suggest to use Colab only if you have issues with your local setup

Environment: cloud setup with JupyterLite

Use JupyterLite:

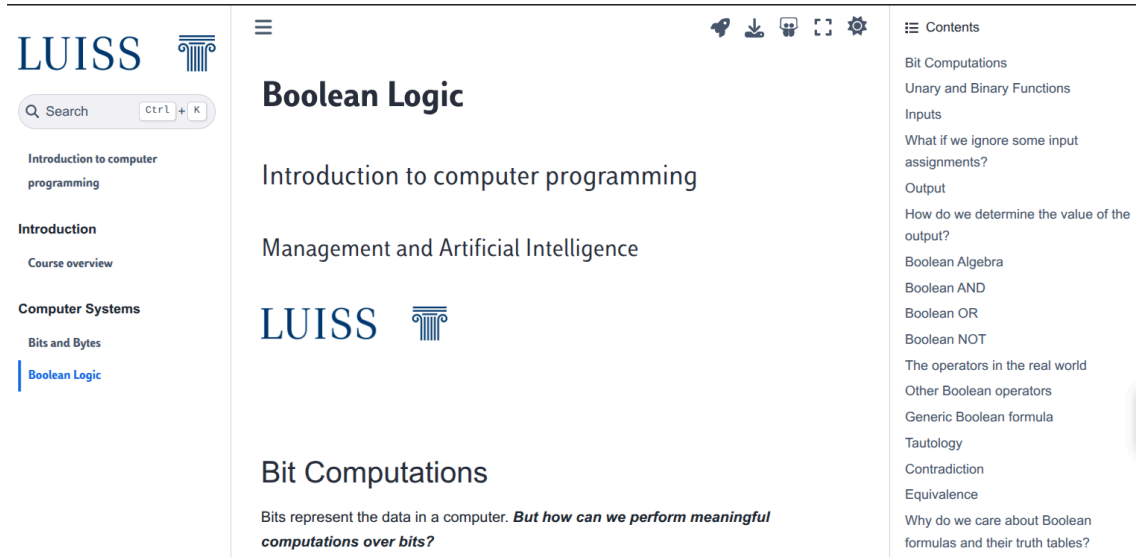
- You can work from any computer. Even your tablet (although, it is a bit inconvenient!)
- Write and execute notebooks (`.ipynb`).
- Everything is saved in the browser's local database
- Everything runs actually on your machine

We suggest to use JupterLite only when you cannot use VSCode and do not want use Colab

Using the book

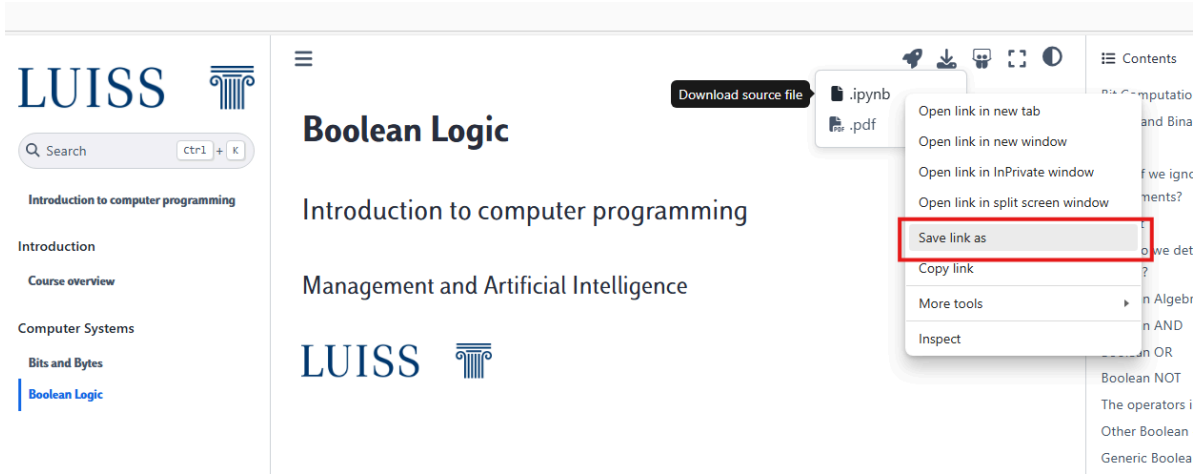
Accessing the material on the book

1. Visit https://introcp.github.io/ to access the book
2. Use the left menu to get the lesson content



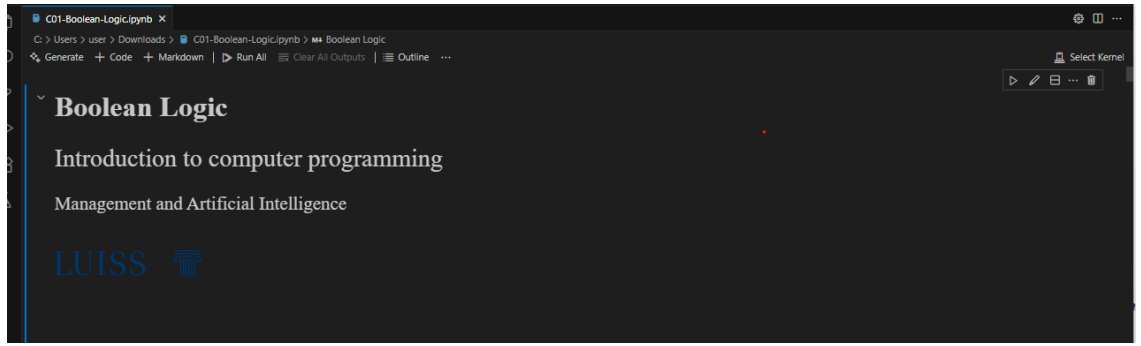
Download notebooks from the book

1. Click on the  icon
2. Right click on **.ipynb > Save link as** to save the file on your pc



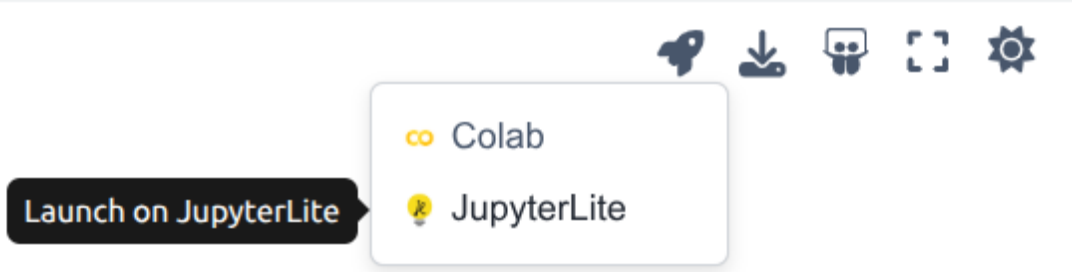
Opening notebooks from the book in VSCode

3. Open the `.ipynb` in VS Code
4. Add your notes and code snippets



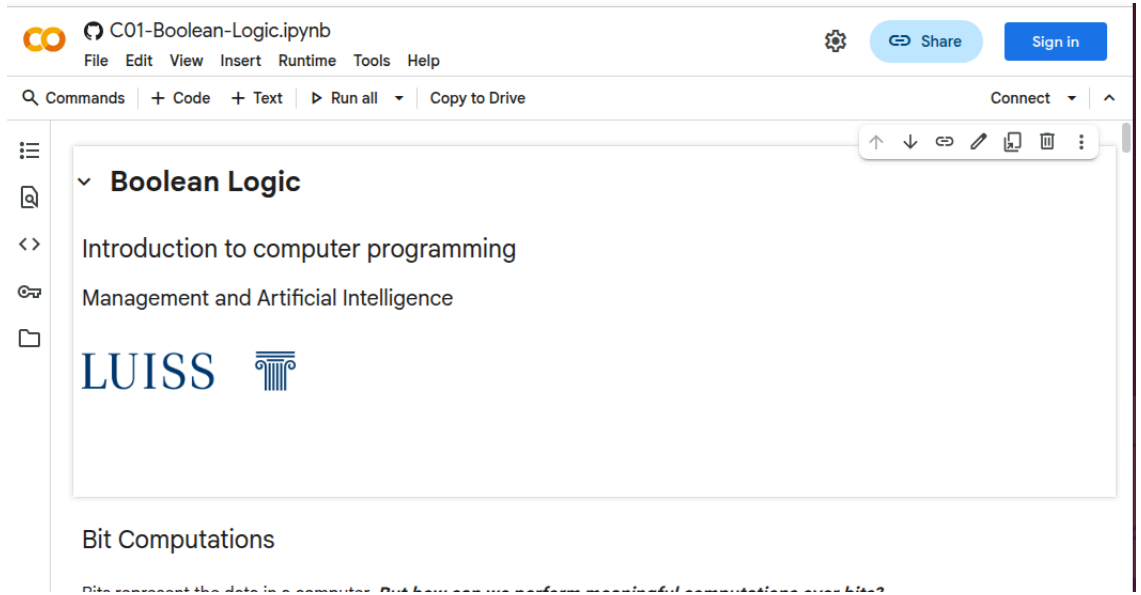
Opening notebooks from the book: Colab or JupyterLite

1. Click on the  icon
2. Click on Colab/JupyterLite



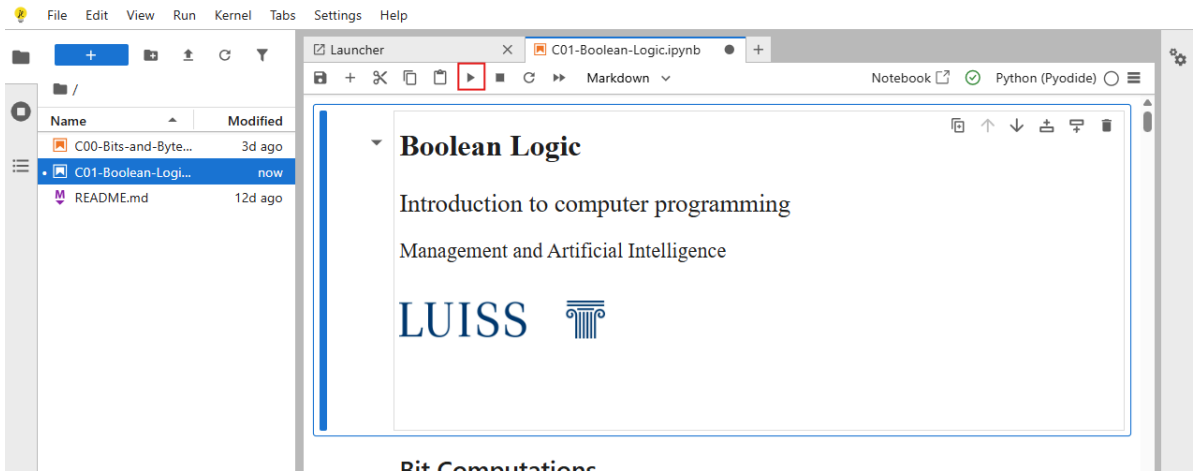
Importing notebooks from the book: Colab

- Import a notebook in colab
- Add your notes or code snippets



Importing notebooks from the book: JupyterLite

- Import the notebook in JupyterLite
- Add your notes or code snippets
- Run code snippets by clicking on the play button (on the top of the notebook or on the left of the code cell)



Importing notebooks from the book: why you should

1. Add your personal notes and thoughts notes to theory slides during the lectures!
2. Work on the Python exercises and run examples without further efforts
3. Active learning (and learning by doing) is a proven way to enhance learning experience and results

Python Virtual Environments

This will be useful for future courses!

Where to install Python packages from the community

We can install Python packages from the community following two strategies:

- ***System-wide***: this may create conflicts between different projects using Python.
    - pros: you need to install a package only once for all projects
    - cons: one project may break due to a package installed/updated for another project

  It is now the deprecated approach.

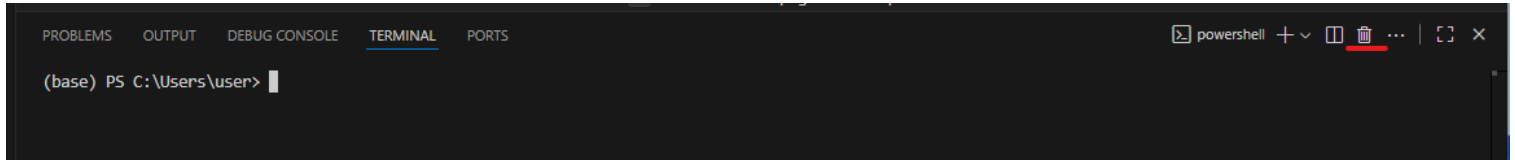
- ***Project-wide***: each project has its own ***Virtual Environment***.
    - pros: no conflicts across packages of different projects
    - cons: a package must be installed again for each project

  This is nowadays the suggested approach and **we will follow it**.

Create a new virtual environment [Only Windows]

1. In VS Code, start the terminal: ***Terminal > New Terminal***

2. Only for Windows: `conda init powershell`
   Then close the terminal



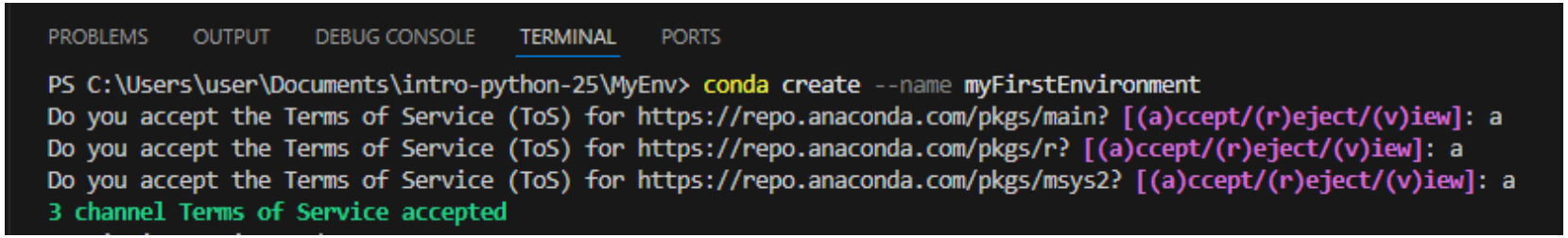
Create a new virtual environment (cont'd)

1. In VS Code, start the terminal: ***Terminal > New Terminal***

2. Type in the terminal: `conda create --name <my-env>`
   Where `<my-env>` is the name of your virtual environment

3. Conda will ask to proceed `Proceed ([y]/n)?`
   Type `y` and hit enter to confirm

NOTE: The first time you create an environment, conda will ask to accept the terms of agreement **before asking to proceed**



Print the environment list

- To verify that the new environment was created correctly, we'll print the list of all the existing environments:
  `conda env list`

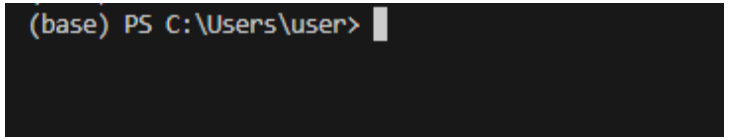
- The newly created environment name should appear in the list

```
PS C:\Users\user\Documents\intro-python-25\MyEnv> conda env list

# conda environments:
#
base                  C:\Users\user\miniconda3
myFirstEnvironment    C:\Users\user\miniconda3\envs\myFirstEnvironment
```

How to use environments while working

- There is a default **base** environment.



- When starting a new project, we will create a new environment

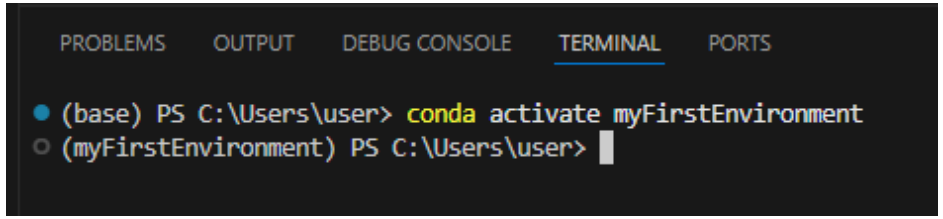
- Then our workflow will be like:

    1. Before starting to edit a project: **activate** the environment created for that project: `conda activate <my-env>`
    2. While working on the project: install any **python package** needed `conda install <package-name>`
    3. When finished: **deactivate** the environment (to avoid conflicts between projects) `conda deactivate`

Activate you environment

1. Type in the terminal: `conda activate <my-env>`
   Where `<my-env>` is the name of your virtual environment

2. The name of the environment will replace of "(base)" at the beginning of the command line, to confirm the environment is active.



Installation of Python packages in the virtual environment

We will use some extra Python bits from the community, including:
`numpy`, `matplotlib`, `seaborn`, `scikit-learn`, and `jupyterlab`.

From VSCode:

1. Start a new terminal using the menu: ***Terminal > New Terminal***
2. `conda install numpy matplotlib seaborn scikit-learn jupyterlab`
3. When prompted to `Proceed ([y]/n)?` type `y` and hit enter. The package download will start